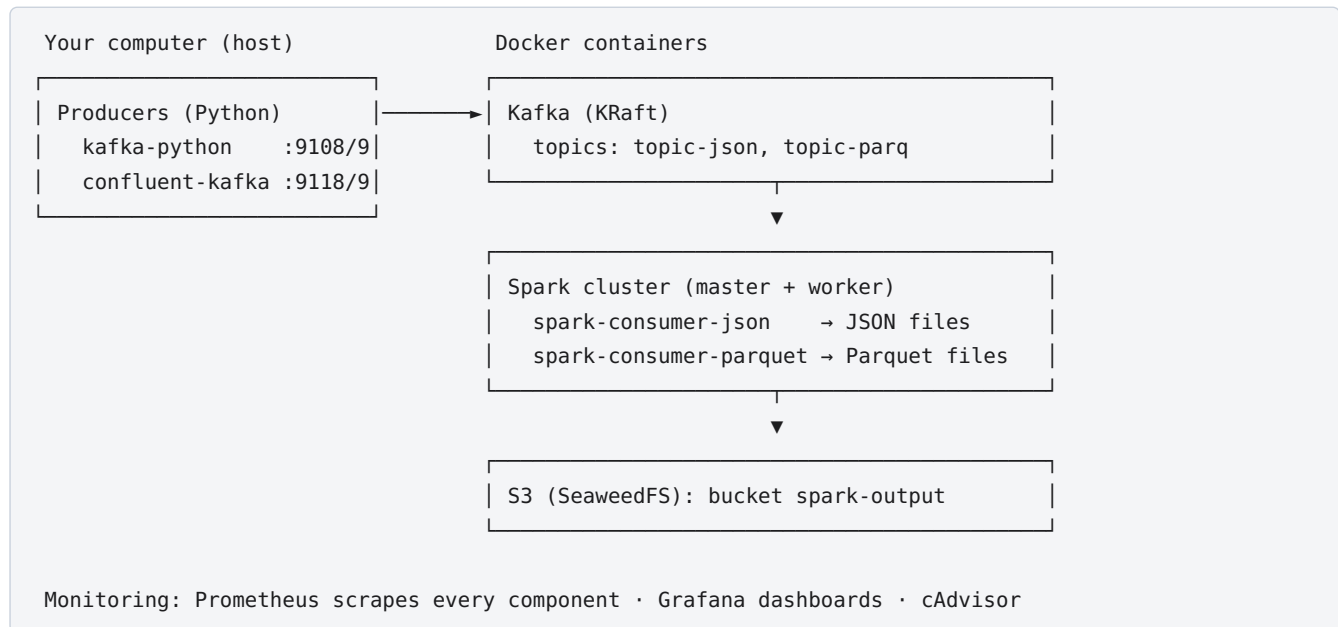


CSE 6332 Streaming Pipeline

Version v2.1.5 · PDF copy of README.md · https://github.com/ToddRosenkrantz/cse6332_project_6 · The online README is always the most current.

Kafka → Spark Structured Streaming → S3, monitored by **Prometheus + Grafana + cAdvisor**.

Runs the same way on **Windows PowerShell**, **Linux bash (native or WSL2)** and **macOS (Intel and Apple Silicon)**. Every image is multi-arch (amd64 + arm64), and all tooling is one Python script: `setup.py`.



What each part does, and how a message travels from producer to S3: see [How the pipeline works](#).

Getting the project

Choose **one** option. Both give you the same files. Git makes later updates a single command; the ZIP needs no extra software.

Option A: Git

You need Git installed (check with `git --version`):

OS	Install Git
Windows	<code>winget install --id Git.Git -e</code> , or the installer from https://git-scm.com/download/win
macOS	<code>xcode-select --install</code>
Linux / WSL2	<code>sudo apt install git</code> (Debian/Ubuntu)

Then, in a terminal (PowerShell, bash or zsh):

```
git clone https://github.com/ToddRosenkrantz/cse6332_project_6.git
cd cse6332_project_6
```

- **A specific release:** `git checkout v2.1.0` (list releases with `git tag`; return to the latest with `git switch main`).

- **Update later:** `python setup.py shutdown`, then `git pull`, then `python setup.py install`. Your `.env` and your data are kept.
- **The previous (v1) project:** `git checkout v1-final`.

Option B: ZIP download (no Git needed)

1. **Download** one of these:

- Latest version: https://github.com/ToddRosenkrantz/cse6332_project_6/archive/refs/heads/main.zip (on GitHub: green **Code** button → **Download ZIP**)
- A specific release: https://github.com/ToddRosenkrantz/cse6332_project_6/archive/refs/tags/v2.1.0.zip (on GitHub: **Tags** → pick a version → **zip**)
- Or the ZIP file your instructor provides

2. **Extract** it somewhere ordinary under your home folder, for example Documents. Don't run it from inside the ZIP preview, and on macOS keep it under your home folder so Docker Desktop can share it.

OS	Command (or right-click → Extract All / double-click)
Windows PowerShell	<code>Expand-Archive .\cse6332_project_6-main.zip -DestinationPath .</code>
macOS / Linux	<code>unzip cse6332_project_6-main.zip</code>

3. **Enter the folder.** Its name depends on what you downloaded: `cse6332_project_6-main`, `cse6332_project_6-2.1.0`, or `cse6332_project_6` for the instructor's ZIP. The name doesn't matter, and you can rename it.

```
cd cse6332_project_6-main
```

- **Update later:** run `python setup.py shutdown` in the old folder, download and extract the new ZIP, copy your `.env` from the old folder into the new one, then run `python setup.py install` in the new folder. Your data is kept (see [Where your data lives](#)), and you can delete the old folder afterwards.

Then continue with the [Installation guide](#): check the prerequisites in Step 1, then carry on from Step 3 inside the project folder.

Installation guide (step by step)

Follow these steps in order. Each step says how to check that it worked before you move on. If a step fails, see [Troubleshooting](#).

Step 1: Check the prerequisites

You need	Check with	Expect	Get it
Docker with Compose v2	<code>docker version</code> and <code>docker compose version</code>	Both print versions; <code>docker version</code> shows a <i>Server</i> section	Windows / macOS: Docker Desktop . Linux: Docker Engine
Python 3.9+ (3.11+ on Windows)	Linux/macOS: <code>python3 --version</code> Windows: <code>python --version</code>	Python 3.9 or newer	python.org or your package manager
Resources for Docker	Docker Desktop → Settings → Resources	≥ 4 CPUs and ≥ 8 GB memory	The stack uses about 6 GB under load
Disk space		≥ 5 GB free	Images (~3 GB), jars (~300 MB), data

Platform notes:

- **Windows:** use Docker Desktop's WSL 2 backend. Docker's memory then comes from WSL, which by default gets half of your RAM. To change it, set `memory=8GB` under `[wsl2]` in `%UserProfile%\wslconfig`, then run `wsl --shutdown`.
- **Linux:** add yourself to the `docker` group so Docker works without `sudo`:
`sudo usermod -aG docker $USER`, then log out and back in. On Debian/Ubuntu, also install `python3-venv` (`sudo apt install python3-venv`).
- **macOS:** keep the project under your home folder, which Docker Desktop shares by default.
- **Docker must be running** (Docker Desktop started) before every `setup.py` command.

Step 2: Get the project

Use Git or the ZIP, as described in [Getting the project](#), and open a terminal **inside the project folder** (the folder containing `setup.py`). Every command below is run from there.

Step 3: Create and activate a Python virtual environment

OS	Create (once)	Activate (in every new terminal)
Linux / WSL2 / macOS	<code>python3 -m venv .venv</code>	<code>source .venv/bin/activate</code>
Windows PowerShell	<code>python -m venv .venv-win</code>	<code>.\.venv-win\Scripts\Activate.ps1</code>

Verify: your prompt starts with `(.venv)` or `(.venv-win)`, and `python --version` shows 3.9 or newer. From here on, `python` means the venv's Python on every OS. You don't need to run `pip` yourself: `install` does it.

Step 4: Run the pre-flight check

```
python setup.py doctor
```

`doctor` checks that Docker and Compose work, Docker's CPUs and memory, the Python packages, that every image has a build for your processor (Intel/AMD or ARM, including Apple Silicon), that the ports are free, and, on macOS, the folder location.

Verify: there are no ✗ lines. Before the *first* install, `doctor` also warns about missing Python packages and ends with ⚠ Doctor: fix the items above.; that's expected, because `install` adds them. After installing, the last line is ✔ Doctor: ready. Fix any ✗ lines (see [Troubleshooting](#)) and run `doctor` again.

Step 5: Install and start everything

```
python setup.py install
```

This takes **5-15 minutes the first time** (mostly downloading Docker images). It:

1. creates `.env` with **unique random passwords and S3 keys** (see [Credentials and security](#))
2. downloads the Spark and Kafka jars and the JMX agent into `volumes/` (~300 MB; files already present are skipped, so re-running resumes)
3. installs the Python packages from `requirements.txt` into your venv
4. starts all containers and waits until every one is healthy
5. creates the Kafka topics (`topic-json`, `topic-parq`, `test-topic`) and the S3 bucket `spark-output`
6. confirms that every Grafana dashboard loaded

Verify: near the end you see `6/6 dashboards available at http://localhost:3000`, followed by the list of **System endpoints**. `install` is safe to run again at any time.

Step 6: Verify that the pipeline works

Run these checks once after installing (and whenever something seems wrong):

#	Command	Success looks like
6a	<code>python setup.py status</code>	Every service line is  . In the Prometheus targets, every line is  except the four <code>kafka-producer-*</code> lines, which show  producer not started until you start producers
6b	<code>python setup.py test</code>	 Event produced and consumed through Kafka.
6c	<code>python setup.py secrets</code>	Your Grafana, SeaweedFS admin and S3 credentials are listed
6d	<code>python setup.py producers start</code>	Four  ... started lines (see Managing producers)
6e	wait about one minute, then <code>python setup.py monitor</code>	Step 3 shows growing Kafka offsets; step 5 shows checkpoint micro-batches; step 6 shows JSON and Parquet files in S3
6f	<code>python setup.py status</code>	Now the <code>kafka-producer-*</code> targets are  too
6g	open http://localhost:3000 → Dashboards → CSE 6332 Pipeline → Pipeline Overview	Producer, Kafka, Spark and S3 graphs show data within a minute

When you're done checking, stop the load with `python setup.py producers stop`.

Step 7: Daily use (stop and start)

When	Command	What happens
You finish working	<code>python setup.py shutdown</code>	Producers stop (flushing their messages), then the Spark consumers, then all containers. All data is kept
You come back	<code>python setup.py up</code>	Everything starts again from where it stopped (1–2 minutes)
Something looks stuck	<code>python setup.py restart <service></code>	Restarts one container, e.g. <code>restart spark-consumer-json</code>

Remember to activate the venv (Step 3) in each new terminal.

Step 8: Uninstall or start fresh

Goal	Command
Remove containers and generated files, keep data	<code>python setup.py clean</code>
Remove everything including all data (Kafka, S3, Grafana, checkpoints)	<code>python setup.py clean --volumes</code>
Start completely fresh afterwards	<code>python setup.py install</code>
Remove the project entirely	<code>python setup.py clean --volumes</code> , then delete the project folder. Docker images can be removed with <code>docker image prune -a</code> (this affects all your Docker images)

`clean` never deletes `.env`, and deletes `results/` only with `--results`.

Quick reference

```
python setup.py doctor      # pre-flight check
python setup.py install    # first-time setup (generates credentials, starts everything)
python setup.py status     # health of every service and Prometheus target
python setup.py secrets    # your logins: Grafana, SeaweedFS admin, S3 keys
python setup.py producers start # send data (see "Managing producers")
python setup.py monitor    # end-to-end check: Kafka offsets, Spark checkpoints, S3 files
python setup.py shutdown   # stop everything, keep data; `python setup.py up` to resume
python setup.py -h         # all commands; `python setup.py <command> -h` for its options
```

Open Grafana at <http://localhost:3000> → **Dashboards** → **CSE 6332 Pipeline** → **Pipeline Overview**. Viewing needs no login; to edit dashboards, sign in with the user and password from `python setup.py secrets`.

 **Every install gets its own random passwords and S3 keys.** There are no shared default passwords. See [Credentials and security](#).

Using setup.py

`setup.py` is the only tool you need. It works the same on Windows, Linux, WSL2 and macOS. Every command has built-in help: `python setup.py -h` lists the commands, and `python setup.py <command> -h` shows a command's options.

Stack lifecycle

Command	Options	What it does and when to use it
<code>install</code>	<code>--no-cadvisor</code>	First-time setup: credentials, jars, Python packages, then <code>up</code> . Safe to re-run; it repairs a partial install
<code>up</code> (alias <code>start</code>)	<code>--no-cadvisor</code>	Start or create all containers and wait until healthy; creates topics and bucket if needed. Use after <code>shutdown</code> , after changing <code>.env</code> , or after pulling a new version
<code>shutdown</code> (alias <code>stop</code>)		Producers → consumers → containers, in that order, so no message is lost. Data is kept
<code>down</code>		Like <code>shutdown</code> , but also removes the containers (data volumes kept). <code>up</code> recreates them
<code>restart</code>	<code>[service]</code>	Restart all containers, or one (e.g. <code>kafka</code> , <code>spark-consumer-parquet</code> , <code>grafana</code>)
<code>clean</code>	<code>--volumes</code> , <code>--keep-jars</code> , <code>--results</code>	Remove containers and everything <code>setup.py</code> created or downloaded (jars, <code>s3.json</code> , logs, run state). <code>--volumes</code> also deletes all data : Kafka, S3 (SeaweedFS), checkpoints, Prometheus, the Grafana database, and volumes left by older versions. <code>.env</code> is always kept

Checking health

Command	Options	What it does and when to use it
<code>doctor</code>		Pre-flight checks: Docker, Compose, resources, Python packages, image architectures, free ports, name clashes, macOS folder sharing
<code>status</code>		Every web endpoint (✓✗) and every Prometheus scrape target. First stop when something looks wrong

Command	Options	What it does and when to use it
monitor		End-to-end data check: topics, offsets, consumer logs, checkpoint progress, newest S3 files
test		Sends one event to <code>test-topic</code> and reads it back: proves Kafka works
ps		Containers with their ID, status and ports (IDs match the cAdvisor dashboard on Docker Desktop)
logs	[service] [--tail N] [-f]	Container logs (last 50 lines by default; <code>-f</code> keeps following new lines until you press Ctrl+C)

Data

Command	Options	What it does and when to use it
s3-ls	[prefix] [--limit N]	List objects in the S3 bucket, newest first, e.g. <code>s3-ls parquet/</code>
audit	[--json FILE]	Exactly-once check (every message in Kafka stored once in S3), file counts and sizes, latency percentiles
compact	[--sink json parquet] [--date D --hour H] [--files N]	Merge one hour of small files into a few and compare read times (see Performance research)
init		Re-create the Kafka topics and the S3 bucket (only needed if you deleted them)
submit	[job]	Run a Spark job from <code>./jobs</code> on the cluster in the foreground (default: a console consumer for debugging)
grafana-export	[uid]	Save dashboards you edited in Grafana into <code>volumes/grafana/dashboards/</code> so they survive a clean install

Settings

Command	Options	What it does and when to use it
secrets	[--rotate]	Show your logins and S3 keys; <code>--rotate</code> replaces them all with new random values on the running stack
config	[--reset]	Show every tuning setting; <code>--reset</code> restores the defaults (credentials and ports untouched)
spark	--trigger, --max-offsets, --cores, --worker-cores, --worker-cpus, --worker-memory, --executor-memory, --driver-memory, --consumer-memory	Show or change Spark tuning. Values are validated, saved to <code>.env</code> , and the affected containers restart automatically
topics	[--partitions N] [--topic T]	Show or increase Kafka topic partitions (Kafka can only increase them)

Load and experiments

Command	Options	What it does and when to use it
producers	see Managing producers	Start, stop and control the data generators
bench	<scenario>, --list, --quick, --label, --steps, ...	Run a scripted experiment; results go to <code>results/</code> (see Performance research)
chaos	kill-consumer pause-kafka restart-s3 stop-worker [json parquet] [--seconds N]	Inject a fault while producers run, and measure recovery

Managing producers

Producers are small Python programs that run on **your computer** (not in Docker) and send IoT-style events to Kafka. Each message is JSON like this:

```
{"msg_id": "ck-topic-json-4211-1790452176-topic-json-p1-57", "device_id": "iot_device_17",  
  "battery_level": 83, "motion_detected": true, "timestamp": "2026-09-26T19:49:34.123456Z"}
```

`msg_id` is unique per message and lets `audit` prove that nothing was lost or duplicated.

Types

	kafka-python (<code>kafka-python</code>)	confluent-kafka (<code>confluent-kafka</code>)
Library	Pure Python	C library (librdkafka) with a Python wrapper; much faster
Senders per topic	1	Several threads (default 2), adjustable while running
Metrics ports (topic-json / topic-parq)	9108 / 9109	9118 / 9119
Compression	none, gzip	none, gzip, snappy, lz4, zstd (default zstd)
Defaults	<code>acks=1</code> , no idempotence	<code>acks=all</code> , idempotent

`producers start` with no options starts **both types for both topics**: four processes.

Count: how many producers

To change	Command	Notes
Which types run	<code>--impl kp</code> , <code>--impl ck</code> or <code>--impl both</code> (default)	on <code>producers start</code>
Which topics	<code>--topic topic-json</code> or <code>--topic topic-parq</code> (default: both)	on <code>producers start</code>
confluent-kafka threads, at start	<code>producers start --impl ck --threads 4</code>	per topic
confluent-kafka threads, while running	<code>producers set-prods topic-json 6</code>	takes effect within 2 seconds

kafka-python always runs one sender per topic; to add kafka-python load, raise its rate.

Rate: how fast they send

Each sender (a kafka-python process or a confluent-kafka thread) sends messages at random, **Poisson-distributed** intervals, with an average of λ (**lambda**) **messages per second**. The load offered to a topic is:

```
offered msgs/s per topic =  $\lambda \times (1 \text{ kafka-python sender} + \text{number of confluent-kafka threads})$ 
```

With the defaults ($\lambda = 5$, 2 threads, both types), each topic receives about $5 \times (1 + 2) = \mathbf{15 \text{ msgs/s}}$.

To change	Command	Notes
λ at start	<code>producers start --rate 50</code>	applies to both topics unless <code>--topic</code> is given
λ while running	<code>producers set-lambda topic-json 100</code>	takes effect within 2 seconds

To change	Command	Notes
Step λ up automatically	<code>producers ramp topic-json --from 10 --to 200 --step 10 --interval 30</code>	runs in the background; <code>producers stop</code> ends it
Maximum speed	<code>producers start --impl ck --burst</code>	ignores λ ; sends as fast as possible

Tuning options (on `producers start`)

Option	Values	Effect
<code>--acks</code>	0, 1, all	How many broker acknowledgments each message waits for
<code>--idempotence</code>	true, false	No duplicates on retry; requires <code>--acks all</code>
<code>--linger-ms</code>	milliseconds	Wait this long to batch messages (throughput vs latency)
<code>--batch-size</code>	bytes	Maximum batch size
<code>--compression</code>	see Types	Compress batches
<code>--payload-bytes</code>	bytes	Add filler to each message to make it bigger
<code>--burst</code>		Send as fast as possible

Invalid combinations are rejected before anything starts, for example `--idempotence true --acks 1`, or `--compression lz4` with `kafka-python`.

Everyday producer commands

Command	What it does
<code>python setup.py producers start [options]</code>	Start in the background (they keep running after you close the terminal)
<code>python setup.py producers status</code>	Each producer: running or not, PID, metrics port, λ , threads and options
<code>python setup.py producers logs topic-json [--impl kp ck] [-n 40]</code>	Last lines of a producer's log
<code>python setup.py producers stop [--impl ...] [--topic ...]</code>	<code>setup.py</code> signals each producer to flush its messages and exit (SIGTERM on Linux/macOS, CTRL_BREAK_EVENT on Windows); a producer that doesn't exit within 15 s is force-stopped

Recipes

```
# 100 msgs/s per topic from confluent-kafka (4 threads x  $\lambda$  25)
python setup.py producers start --impl ck --threads 4 --rate 25

# one light kafka-python producer on one topic
python setup.py producers start --impl kp --topic topic-json --rate 10

# speed up one topic while running
python setup.py producers set-lambda topic-parq 200

# tuned confluent-kafka producers with bigger messages
python setup.py producers start --impl ck --acks 1 --compression lz4 --payload-bytes 500

# stop only the kafka-python producers
python setup.py producers stop --impl kp
```

Where producer state lives

File or folder	Contents
lambda_<topic>.txt	Current λ for that topic, re-read by all its senders every 2 s
producers_<topic>.txt	Current confluent-kafka thread count for that topic
run/	PID file and options of each running producer
logs/	Each producer's output (<code><impl>_<topic>.out</code>) and kafka-python's per-run CSV of rates

Their metrics appear in Prometheus (the four `kafka-producer-*` targets) and on the **Kafka Producers** and **Pipeline Overview** dashboards.

How the pipeline works

The stack is a small copy of a typical cloud **streaming data platform**: devices or apps publish events to a durable message log, a distributed processing engine turns the stream into analytics-ready files in object storage, and a monitoring layer watches every component. Each box in the diagram at the top is one or more Docker containers.

What each container does

Container	Role in this pipeline	Real-world use or cloud equivalent
Producers (Python, on your computer, not in Docker)	Simulate IoT devices: generate JSON events at a chosen rate and send them to Kafka. Each exposes its own metrics for Prometheus	Phones, web apps and sensors sending telemetry, e.g. connected devices reporting through AWS IoT Core
kafka	The message broker (Apache Kafka 3.9 in KRaft mode, so it manages its own metadata). Stores events durably in two topics, <code>topic-json</code> and <code>topic-parq</code> , so producers and consumers never need to be up at the same time or run at the same speed. A built-in JMX agent exports the broker's internal metrics	Kafka was created at LinkedIn to move activity events between systems; Netflix and Uber run large Kafka deployments for real-time event streams. Cloud: Amazon MSK, Confluent Cloud, Azure Event Hubs
kafka-init	One-shot job at start-up: creates the topics, then exits	Infrastructure-as-code provisioning (e.g. Terraform creating MSK topics)
kafka-exporter	Reads topic and partition offsets from Kafka and publishes them as Prometheus metrics	Standard way to monitor Kafka throughput and backlog in production
spark-master	Apache Spark's cluster manager : tracks the workers and hands out CPU cores and memory to applications	The role YARN or Kubernetes plays on Amazon EMR , Google Dataproc and Databricks
spark-worker	Runs the executors : the processes that actually read, parse and write the data, in parallel	The worker nodes/VMs of an EMR or Databricks cluster
spark-consumer-json , spark-consumer-parquet	Two independent Spark Structured Streaming applications (the <i>drivers</i>). Each reads one topic, turns every message into a table row, and writes it to S3: one as JSON, one as Parquet. Each keeps a checkpoint of how far it has read	Streaming ETL into a data lake, the pattern behind clickstream and IoT pipelines. Spark began at UC Berkeley's AMPLab ; Netflix and Uber use it for large-scale ETL and machine learning
seaweedfs	S3-compatible object storage holding the output bucket <code>spark-output</code> . Includes the S3 API, a filer (file browser) and volume servers	SeaweedFS is based on Facebook's Haystack design for storing billions of photos. Cloud: Amazon S3 , Google Cloud Storage, Azure Blob (Netflix, for example, keeps its data warehouse in S3)

Container	Role in this pipeline	Real-world use or cloud equivalent
<code>seaweedfs-admin</code>	Web console for the storage: buckets, S3 users and keys, topology, file browser	The AWS S3 and IAM consoles
<code>s3-init</code>	One-shot job at start-up: creates the S3 bucket, then exits	Provisioning scripts / infrastructure-as-code
<code>prometheus</code>	Monitoring database: every few seconds (2 s for producers, 5 s for most services, 10 s for cAdvisor) it scrapes metrics from the producers, Kafka, Spark, SeaweedFS and cAdvisor, and stores them as time series	Created at SoundCloud ; the standard monitoring system for Kubernetes. Cloud: Amazon Managed Service for Prometheus, CloudWatch
<code>grafana</code>	Dashboards: queries Prometheus and draws the graphs; also shows the markers <code>bench</code> and <code>chaos add</code>	The usual dashboard layer on top of Prometheus in Kubernetes monitoring stacks. Cloud: Amazon Managed Grafana, CloudWatch dashboards
<code>cadvisor</code>	Measures CPU, memory and network use of every container	Created by Google and built into Kubernetes (the kubelet uses it to report container resource usage)

The message processing path

1. **Produce.** A producer creates an event such as `{"msg_id": "...", "device_id": "iot_device_17", "battery_level": 83, "motion_detected": true, "timestamp": "..."}` and sends it to Kafka at `127.0.0.1:9092`, on `topic-json` or `topic-parq`.
2. **Store durably.** Kafka appends the event to the end of the topic's log (its **partition**), gives it the next **offset** number, and acknowledges it to the producer (`acks`). From this point the event is safe even if nothing is reading it yet.
3. **Plan a micro-batch.** Each Spark consumer works in small **micro-batches**. At the start of each one, the driver asks Kafka for the newest offsets and records the planned range ("offsets 1,200-1,450") in its checkpoint.
4. **Process in parallel.** Executors on the Spark worker read that range from Kafka, parse the JSON, and add columns: `date` and `hour` (for partitioning the output) and `processed_at` (for latency measurements).
5. **Write to S3.** The executors write the rows as files to `s3a://spark-output/json/...` or `s3a://spark-output/parquet/...`, organized as `date=.../hour=...` folders. JSON is readable text; Parquet is a compressed, columnar format built for analytics.
6. **Commit.** Only after the files are written does Spark record them in the sink's `_spark_metadata` log and mark the batch as committed in its checkpoint. If anything crashes mid-batch, Spark redoes that batch from the checkpoint and readers ignore the uncommitted files. That's how each message ends up in S3 **exactly once** (what python `setup.py audit` checks).

Monitoring runs alongside: every component exposes metrics over HTTP, Prometheus collects them every few seconds, and Grafana visualizes them. That's how you can see where the pipeline slows down: in the producers, Kafka, Spark or storage.

Web UIs and ports

Every port is set in `.env`; the defaults are shown. `python setup.py up` prints these links, and `python setup.py status` checks each one.

Browser UIs

Container	UI	URL	Login
grafana	Dashboards	http://localhost:3000	Viewing: none (read-only). Editing: admin + generated password (python setup.py secrets)
prometheus	Targets, queries, graphs	http://localhost:9090/targets	none
seaweedfs-admin	SeaweedFS admin console: buckets, S3 users and keys, policies, topology, file browser, maintenance	http://localhost:23646	admin + generated password (python setup.py secrets)
seaweedfs	Filer: browse, upload and download files in the bucket	http://localhost:8888/buckets/spark-output/	none
seaweedfs	Master: cluster status, volumes, topology	http://localhost:9333	none
seaweedfs	Volume server status	http://localhost:8089/ui/index.html	none
spark-master	Cluster: workers, running apps, cores	http://localhost:8080	none
spark-worker	Worker: executors, resources	http://localhost:8081	none
spark-consumer-json	JSON sink app. Structured Streaming tab has rates and batch durations	http://localhost:4041/StreamingQuery/	none
spark-consumer-parquet	Parquet sink app	http://localhost:4042/StreamingQuery/	none
cadvisor	Per-container CPU and memory	http://localhost:8088/containers/	none

APIs and metrics endpoints (no UI)

Container / process	Port	Purpose
kafka	9092	Kafka broker for host clients (advertised as 127.0.0.1:9092)
kafka	9094	Kafka for clients on other machines (advertised as KAFKA_LAN_HOST ; see Accessing the stack from other machines)
kafka	7071	JMX exporter: http://localhost:7071/metrics
kafka-exporter	9308	Topic, partition and offset metrics: http://localhost:9308/metrics
seaweedfs	8333	S3 API (path-style; generated access and secret keys from python setup.py secrets)
seaweedfs	9327	SeaweedFS metrics: http://localhost:9327/metrics
spark-master	7077	Spark cluster RPC (spark://localhost:7077)
spark-consumer-*	4041 / 4042	Also serve /metrics/driver/prometheus and /metrics/executors/prometheus
host producers	9108, 9109	kafka-python producer metrics (topic-json, topic-parq)
host producers	9118, 9119	confluent-kafka producer metrics (topic-json, topic-parq)

`kafka-init` and `s3-init` are one-shot setup containers and publish no ports.

Grafana dashboards

Dashboards are **created automatically**: Grafana provisions every JSON file in `volumes/grafana/dashboards/` into a fresh database on start, so there's no `grafana.db` to restore and no manual import. `install` and `up` finish by confirming that each dashboard loaded (for example "6/6 dashboards available"). To add a dashboard, drop its JSON into that folder with `"uid"` set and `datasource uid` `aetbs3rckegowe`.

Dashboard	Shows
Pipeline Overview	Producers → Kafka broker → Spark (input vs processed rate, batch latency, cores) → S3 bucket
Kafka Cluster Overview	Brokers, topics, offsets, ISR, Spark consumption, broker JVM
Kafka Exporter Overview	kafka-exporter topic and partition metrics
Kafka Producers (custom)	λ , message rate, delay, errors per producer
SeaweedFS (S3 storage)	Official SeaweedFS dashboard. Cluster, replication and erasure-coding panels stay empty on this single-node store
Containers (cAdvisor)	CPU and memory per container

UI edits persist in Grafana. To make one permanent, run `python setup.py grafana-export`.

Accessing the stack from other machines

By default everything is used from the machine running Docker. The web UIs are already published on every network interface, so other machines on your LAN can reach them once your firewall allows it. Kafka needs one `.env` setting.

1. Find this machine's LAN address

OS	Command
Windows	<code>ipconfig</code> , then the "IPv4 Address" of your Wi-Fi or Ethernet adapter
macOS	<code>ipconfig getifaddr en0</code> (Wi-Fi; try <code>en1</code> if empty)
Linux	<code>hostname -I</code>

Other machines then use `http://<LAN-IP>:3000` for Grafana, and the same ports as in [Web UIs and ports](#) for the rest. A LAN IP from DHCP can change when you switch networks.

2. Open the firewall

Setup	What blocks inbound connections	What to do
Windows, WSL2 NAT mode (the default)	Windows Defender Firewall	Allow Docker Desktop Backend on <i>Private</i> networks: Windows Security → Firewall → <i>Allow an app through firewall</i>
Windows, WSL2 mirrored mode (<code>networkingMode=mirrored</code> in <code>%UserProfile%\wslconfig</code>)	Hyper-V firewall for WSL (blocks all inbound by default)	Run the command below in an Administrator PowerShell

Setup	What blocks inbound connections	What to do
macOS	macOS firewall, if it's turned on	Allow Docker when prompted, or add it under System Settings → Network → Firewall → Options
Linux	Usually nothing	Docker adds its own iptables rules, and those bypass <code>ufw</code> , so published ports are reachable from the LAN by default

Windows with WSL mirrored mode: allow the UI ports (add `9094` if you also want LAN Kafka clients):

```
New-NetFirewallHyperVRule -Name "CSE6332-UIs" -DisplayName "CSE 6332 pipeline web UIs" `
  -Direction Inbound -Action Allow -Protocol TCP `
  -VMCreatorId '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}' `
  -LocalPorts 3000,9090,23646,8888,9333,8089,8080,8081,4041,4042,8088
# remove again with: Remove-NetFirewallHyperVRule -Name "CSE6332-UIs"
```

Check what's currently allowed with `Get-NetFirewallHyperVVMSetting -PolicyStore ActiveStore -Name '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}'`.

In mirrored mode, Windows can't reach the stack through its own LAN IP (connections to itself are not routed into WSL). Test from another device, or use `127.0.0.1` locally.

3. Kafka clients on other machines (optional)

Kafka tells every client which address to reconnect to, so remote clients need an address that points back to this machine:

1. In `.env`, set `KAFKA_LAN_HOST=<LAN-IP or DNS name>`. `KAFKA_LAN_PORT` defaults to `9094`.
2. Run `python setup.py up`. The endpoint list then shows Kafka for LAN clients : `<LAN-IP>:9094`.
3. On the remote machine, use `--bootstrap <LAN-IP>:9094` (for example, a producer script copied there).

Clients on this machine keep using `127.0.0.1:9092`, and containers keep using `kafka:29092`. If the LAN IP changes, update `.env` and run `up` again.

Security before you open ports

Opening the firewall exposes the stack to everyone on that network:

- Your passwords and S3 keys are unique random values (see [Credentials and security](#)). If you have shared them or suspect they leaked, run `python setup.py secrets --rotate`. Grafana dashboards are viewable read-only without a login; set `GRAFANA_ANONYMOUS_ACCESS=false` in `.env` to require a login even to view.
- Prometheus, the Spark UIs, cAdvisor and the SeaweedFS filer/master pages have **no login**. The filer page can upload and delete files. Open only the ports you need, and only on trusted networks (*Private*, not *Public*).
- Kafka's LAN listener is unauthenticated plaintext.

Performance research

The capstone experiment is driven entirely by `setup.py`, so there are no configuration files to edit. Pick one research avenue from [docs/research_projects.md](#), change **one setting**, and compare it with a default run:

Avenue	Question	Scenario
A	Where does ingestion saturate, and which producer settings move the ceiling?	A_ingestion, A_ingestion_burst
B	What rate can each Spark sink sustain, and how does it scale?	B_spark_scaling
C	How do triggers and formats create small files, and what does compaction gain?	C_small_files + compact
D	How does end-to-end latency change with load and batching?	D_latency
E	Is data lost or duplicated under failures, and how fast is recovery?	E_faults + chaos

```
python setup.py bench B_spark_scaling --label default           # default settings
python setup.py bench B_spark_scaling --worker-cpus 0.5 --label changed # one setting changed
python setup.py config --reset                                   # back to defaults
python setup.py bench B_spark_scaling --quick                   # 30-second steps, for a
trial run
```

Each run writes `results/<date>_<scenario>_<label>/` containing:

- `report.md`: per-step table, sustainable throughput, latency p50/p95/p99, exactly-once accounting, resource usage and a Grafana link
- `steps.csv`, `config.json` and `audit.json`

Steps and faults appear as markers on the Grafana dashboards.

Further reading

- [README.pdf](#): a printable copy of this README (the online version is always the most current)
- [docs/research_projects.md](#): the capstone research avenues, commands, and what to submit
- [docs/report_examples_and_rubric.md](#): example executive summary and abstract, and the grading rubric
- [docs/producers_dual_stack.md](#): running and comparing both producer implementations
- [docs/producer_tuning.md](#): assignment framework for producer tuning experiments

Configuration: `.env`

All

credentials

(S3 keys, Grafana admin, SeaweedFS admin), ports, image tags and Spark resources live in `.env`, and `docker-compose.yaml` contains no literal values. `install` and `up create` `.env` from `.env.example` if it's missing. When a newer version of the project adds settings, they also append the missing keys to your existing `.env` and keep your own values.

- **Spark resources**: there's one Spark application per sink (JSON and Parquet), just like independent jobs on a cloud cluster. Each app takes `SPARK_APP_CORES`, so `SPARK_WORKER_CORES` must be at least twice that. `doctor` checks this.
- **cAdvisor** is on by default. To save memory and CPU, use `up --no-cadvisor` for one run, or set `CADVISOR_ENABLED=false`.

Credentials and security

No default passwords. The first time `install` (or `up`) creates `.env`, it generates unique random credentials with Python's `secrets` module:

Credential	Generated value	Used by
<code>S3_ACCESS_KEY</code>	20 characters (AWS style, <code>AK...</code>)	Spark sinks, <code>audit</code> , <code>s3-ls</code> , S3 clients
<code>S3_SECRET_KEY</code>	40 random letters and digits	the same
<code>GRAFANA_ADMIN_PASSWORD</code>	24 random letters and digits	Grafana login (user <code>admin</code>)
<code>SEAWEED_ADMIN_PASSWORD</code>	24 random letters and digits	SeaweedFS admin console (user <code>admin</code>)

- **See them:** `python setup.py secrets` prints every login and key, with its URL.
- **Change them:** `python setup.py secrets --rotate` generates new values and applies them to the running stack. It restarts SeaweedFS, its console and the Spark sinks, resets Grafana's password inside its database, and verifies the new logins. Don't edit these passwords by hand: Grafana keeps its admin password in its own database, so editing `.env` alone would lock you out.
- **Where they live:** only in your `.env`. It's gitignored (never committed or zipped) and readable only by you on Linux and macOS. `.env.example` holds placeholders only.
- **Older installs:** if your `.env` still has default or placeholder values (such as `admin`), every command warns you until you run `secrets --rotate`.
- **Grafana:** anyone who can reach it may *view* dashboards read-only, which is standard practice. Editing, settings and the admin API always require the login. Set `GRAFANA_ANONYMOUS_ACCESS=false` in `.env` to require a login even to view.

Not protected by a password (fine on your own computer; think before opening ports to a network): Prometheus, the Spark UIs, cAdvisor, the SeaweedFS filer/master/volume pages, and Kafka (plaintext, no authentication). See [Accessing the stack from other machines](#).

Where your data lives

Everything the containers store (Kafka messages, S3 objects, Spark checkpoints, Prometheus history, Grafana settings) is kept in Docker **named volumes**, not in the project folder:

`cse6332_kafka_data`, `cse6332_seaweedfs_data`, `cse6332_seaweedfs_admin`, `cse6332_spark_checkpoints`, `cse6332_prometheus_data`, `cse6332_grafana_data`

Action	Data
<code>shutdown</code> , <code>down</code> , <code>up</code> , restarting Docker or the computer, upgrading images	kept
Renaming, moving or re-cloning the project folder	kept: the Compose project name is fixed (name: <code>cse6332</code>), so the volume names don't depend on the folder
<code>python setup.py clean --volumes</code> , <code>docker compose down -v</code> , <code>docker volume prune</code> , Docker Desktop "Clean / Purge data" or reset	deleted

Only one copy of the stack can run at a time. Starting it from a different folder takes over the same containers and data.

Troubleshooting

Start with these three commands: they diagnose most problems.

```
python setup.py doctor      # setup problems: Docker, resources, ports, packages, name clashes
python setup.py status      # which service or Prometheus target is down
python setup.py logs <service> --tail 100  # why it is down (e.g. logs spark-consumer-json)
```

`up` also repairs several problems automatically before starting (missing or root-owned folders, unreadable files, missing `.env` settings).

Installing

Problem	Cause	Fix
<code>docker: command not found</code> , or <code>Cannot connect to the Docker daemon</code>	Docker not installed or not running	Install Docker Desktop / Docker Engine and start it; wait until it reports "running", then retry
<code>permission denied ... /var/run/docker.sock</code> (Linux)	Your user isn't in the <code>docker</code> group	<code>sudo usermod -aG docker \$USER</code> , then log out and back in
<code>docker compose</code> not found, or <code>doctor</code> says Compose v2 is missing	Old Docker with only <code>docker-compose</code> (v1)	Update Docker Desktop, or install the Docker Compose plugin
<code>Activate.ps1</code> cannot be loaded because running scripts is disabled (Windows)	PowerShell execution policy	Run once: <code>Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned</code> , then activate again
The virtual environment was not created successfully because <code>ensurepip</code> is not available (Debian/Ubuntu)	<code>python3-venv</code> missing	<code>sudo apt install python3-venv</code> (or <code>python3.X-venv</code> for your version)
<code>setup.py</code> needs Python 3.9 or newer	Old Python in the <code>venv</code>	Install a newer Python, delete the <code>venv</code> folder, create it again (Step 3)
Downloads fail or stall during <code>install</code>	Network, proxy or firewall	Check your connection and run <code>install</code> again; finished downloads are kept, and partial ones are retried
<code>doctor: Port NNNN (..._PORT)</code> is already in use	Another program uses that port (often another Grafana, Jupyter, or the old project)	Stop that program, or pick another port in <code>.env</code> (e.g. <code>GRAFANA_PORT=3001</code>) and run <code>up</code>
<code>doctor: has no linux/arm64 build</code>	An image doesn't support your processor	Report it; all default images support Intel/AMD and ARM (Apple Silicon)
macOS: mounts denied	The project is outside Docker Desktop's shared folders (<code>/Users</code> , <code>/Volumes</code> , <code>/private</code> , <code>/tmp</code> , <code>/var/folders</code>)	Move the project under your home folder, or add its folder in Docker Desktop → Settings → Resources → File sharing

Starting the stack

Problem	Cause	Fix
Containers from another stack use the same names	Another copy of the project, or the old <code>cse6332_project</code> , is running	Run the exact <code>docker compose down</code> or <code>docker rm -f ...</code> command printed by <code>up</code>

Problem	Cause	Fix
up times out waiting for a container, or a container keeps restarting	Usually not enough memory for Docker	<code>python setup.py logs <service></code> ; give Docker more memory (Step 1); try <code>python setup.py up --no-cadvisor</code>
Spark/JMX jars are missing	The jars were removed (e.g. by <code>clean</code>)	<code>python setup.py install</code>
A folder named <code>volumes/seaweedfs/s3.json</code> , or root-owned folders under <code>volumes/</code>	<code>docker compose up</code> was run before <code>setup.py</code>	Just run <code>python setup.py up</code> ; it repairs this automatically (no <code>sudo</code> needed)
Container errors reading config files after editing them on Windows	The file was saved with Windows (CRLF) line endings	Save it with LF line endings (most editors show this in the status bar); <code>.gitattributes</code> keeps Git checkouts LF
cAdvisor fails to start (macOS)	It needs host system folders macOS doesn't share	Safe to ignore: <code>up</code> warns and continues. Or use <code>python setup.py up --no-cadvisor</code>

Producers

Problem	Cause	Fix
Kafka is not reachable at 127.0.0.1:9092	The stack isn't running	<code>python setup.py up</code>
A producer exited immediately	See its log: <code>logs/<impl>_<topic>.out</code>	If <code>setup.py</code> says metrics port busy : another producer is using it (<code>producers status</code>), or see the WSL2 mirrored-networking row below
<code>--idempotence true</code> requires <code>--acks all</code> , or <code>kafka-python</code> supports <code>--compression none gzip</code>	Invalid option combination	Use a valid combination (see Managing producers)
Produced rate is lower than $\lambda \times$ senders	The producer or your CPU is the bottleneck, or (Windows, Python < 3.11) coarse timers	Use <code>confluent-kafka</code> (<code>--impl ck</code>), fewer threads, or Python 3.11+ on Windows
<code>producers stop</code> reports force-stopped	A producer didn't finish flushing within 15 s (e.g. Kafka was down)	Harmless; messages it had not yet sent are lost. Raise <code>PRODUCER_STOP_TIMEOUT</code> in <code>.env</code> if it happens often

Data not arriving in S3

Problem	Cause	Fix
<code>monitor</code> step 5 says No checkpoint yet, or step 6 shows no files	The Spark consumers are still starting (up to a minute), or no producers are running	Wait a minute; check <code>producers status</code> ; then <code>python setup.py logs spark-consumer-json</code>
Kafka offsets grow but S3 files don't	A Spark consumer is failing or out of memory	<code>python setup.py logs spark-consumer-json</code> ; check <code>python setup.py spark settings</code> ; <code>python setup.py config --reset</code> restores the defaults
<code>audit</code> reports FAIL	Messages still in flight, or data was deleted	Stop producers, wait until <code>monitor</code> shows the sinks caught up, run <code>audit</code> again
Spark settings changed and consumers are slow to start	A small <code>--worker-cpus</code> makes Spark start slowly (minutes)	Wait, or <code>python setup.py config --reset</code>

Logins and dashboards

Problem	Cause	Fix
Don't know the Grafana or SeaweedFS password	They are generated at install	<code>python setup.py secrets</code>
Grafana rejects the password from <code>secrets</code>	<code>.env</code> was replaced or edited by hand after Grafana was set up	<code>python setup.py up</code> , then <code>python setup.py secrets --rotate</code> (resets Grafana's password to a new value)
<code>.env</code> uses default or placeholder credentials warning	An older <code>.env</code>	<code>python setup.py secrets --rotate</code>
Dashboards show "No data"	Nothing is producing data yet, or the time range is too old	Start producers; set the time picker to "Last 15 minutes"
Containers (cAdvisor) panels empty	cAdvisor disabled or not supported on your platform	Expected with <code>--no-cadvisor</code> ; on Docker Desktop containers appear by 12-character ID (match them with <code>python setup.py ps</code>)
A dashboard you edited is gone after <code>clean --volumes</code>	Edits live in Grafana's database	Save them first with <code>python setup.py grafana-export</code>

Platform-specific

- **Windows and localhost**: host-side tools connect to `127.0.0.1`, because Windows resolves `localhost` to IPv6 `::1` first. With WSL2 **mirrored** networking, `::1` isn't bridged between Windows and WSL (a documented WSL limitation), so a `localhost` connection hangs instead of falling back. Keep `KAFKA_BOOTSTRAP_HOST=127.0.0.1:9092`. Browsers fall back to IPv4 automatically, so `http://localhost:...` links work.
- **WSL2 with `networkingMode=mirrored`**: after running producers from Windows, WSL can't reuse ports 9108–9119 for a few minutes (and vice versa). Run producers from one side at a time.
- **Docker Desktop and WSL share one engine**. Stop the stack from one side (`python setup.py down`) before starting it from the other.
- **Windows needs its own venv**. A venv created in WSL doesn't work from PowerShell, and vice versa (hence `.venv-win`).
- **Restrictive umask (Linux/macOS, e.g. 077)**: containers run as their own users and must be able to read the mounted files. `up` makes them readable automatically.

Start completely fresh

If nothing else helps, reset the project (this **deletes all data**; `.env` and `results/` are kept):

```
python setup.py clean --volumes
python setup.py install
```

Changes from `cse6332_project`

Before	Now
Bitnami Kafka plus a separate coordination service	Apache Kafka 3.9 in KRaft mode (Kafka manages its own metadata)
A separate S3 object store and web console (its community images are no longer published)	SeaweedFS S3 gateway + SeaweedFS admin console (:23646)

Before	Now
Makefile + bash scripts, <code>wget</code> , <code>nc</code> , a host-side S3 client	<code>setup.py</code> (Python stdlib + pip packages), init containers for topics and bucket
<code>grafana.db</code> restore	Dashboards provisioned from JSON
Fixed default passwords in compose and jobs	Unique random credentials generated at install, stored only in an owner-only <code>.env</code> ; <code>secrets --rotate</code>
Spark 3.5.5 (Bitnami)	Apache Spark 3.5.9, with jar versions matched to its pom
Checkpoints in <code>/tmp</code> (lost on restart)	Checkpoints in a named volume

Superseded files are kept in `retired/`, which is gitignored.